

Doc. no.:	P0882R0
Date:	2017-12-30
Audience:	LEWG
Reply-to:	Yonggang Li <gnagnoyil at gmail dot com>

User-defined Literals for `std::filesystem::path`

Introduction

This paper proposes three sets of user-defined literals for `std::filesystem::path` (operator `"" native` , operator `"" generic` and operator `"" path`), each of which has the same effect to the constructor call with one particular format, respectively. Therefore those user-defined literals do not only create `"" path` literals," but also act as “named constructors.”

Motivation

C++17 introduces a new filesystem library containing a class `std::filesystem::path` to represent a file/directory path value. Currently to create a prvalue of `std::filesystem::path` from a narrow/wide/u16/u32 string literal, users have to explicitly call the constructors, making the code unnecessary verbose. In particular, if a user wants to create a prvalue of `std::filesystem::path` in specific path format, the user has to specify it by setting the last argument to the constructor call, making the code even more verbose:

```
namespace fs = std::filesystem;
fs::path foo(fs::path, fs::path);
fs::path bar(fs::path);
fs::path fileloc = foo(fs::path("/usr", fs::path::generic_format), bar(fs::path("/
```

One possible workaround is to use `auto_format` , the default value for the format parameter, instead of explicitly setting it. However, that would cause the program to detect the format at runtime, resulting in a runtime overhead, which is unnecessary for users that do not need auto

format detection. Therefore such workaround is a bit against zero-overhead abstraction and “You don’t pay for what you don’t need” principle.

The above dilemmas can be easily solved if we introduce two set of user-defined literals operator `""` generic and operator `""` native in a new inline namespace `std::filesystem::literals` , each of which is responsible for creating an `std::filesystem::path` object of one specific path format respectively. The code above can then be rewritten like this:

```
std::filesystem::path foo(std::filesystem::path, std::filesystem::path);
std::filesystem::path bar(std::filesystem::path);
using namespace std::filesystem::literals;
std::filesystem::path fileloc = foo("/usr"generic, bar("/include/stdio.h"native));
```

Here the user-defined literal works like “named constructor” in other languages such as Vala.

To improve orthogonality, we also add another set of user-defined literals operator `""` path being responsible for creating an `std::filesystem::path` object with path format set to `auto_format` .

Discussion

UTF-8 string literal issues

The current standard uses a different approach to create a path object from a UTF-8 string literal: `std::filesystem::u8path` . It would be ideal if `std::filesystem::path` object could be constructed through a UTF-8 string literal just like the others. But this is not possible because UTF-8 string literals and narrow string literals share the same type; therefore it cannot be distinguished in the declarations of user-defined literals. Furthermore

`std::filesystem::u8path` does not accept a path format argument. From the reasons above we decided not to add anything that directly has something to do with UTF-8 string literals before the Standard Committee get a solution about the type of UTF-8 string literals.

Namespace to put

By far all user-defined literals in the standard library are defined within namespace `std::literals` , and all the types of the objects those user-defined literals constructs are defined directly in namespace `std` . This proposal, however, suggests to put

`std::filesystem::path` literals in an inline namespace called `std::filesystem::literals` , to avoid any potential name conflicts in the future.

Possible Implementation

```
namespace std::filesystem{
    inline namespace literals{
        path operator "" generic(const char *str, size_t len){
            return path(str, str + len, path::generic_format);
        }
        path operator "" generic(const wchar_t *str, size_t len){
            return path(str, str + len, path::generic_format);
        }
        path operator "" generic(const char16_t *str, size_t len){
            return path(str, str + len, path::generic_format);
        }
        path operator "" generic(const char32_t *str, size_t len){
            return path(str, str + len, path::generic_format);
        }

        path operator "" native(const char *str, size_t len){
            return path(str, str + len, path::native_format);
        }
        path operator "" native(const wchar_t *str, size_t len){
            return path(str, str + len, path::native_format);
        }
        path operator "" native(const char16_t *str, size_t len){
            return path(str, str + len, path::native_format);
        }
        path operator "" native(const char32_t *str, size_t len){
            return path(str, str + len, path::native_format);
        }

        path operator "" path(const char *str, size_t len){
            return path(str, str + len);
        }
        path operator "" path(const wchar_t *str, size_t len){
            return path(str, str + len);
        }
        path operator "" path(const char16_t *str, size_t len){
            return path(str, str + len);
        }
        path operator "" path(const char32_t *str, size_t len){
            return path(str, str + len);
        }
    }
}
```

Proposed Wording

The wording is relative to N4713.

Insert the following in 30.11.5 [fs.filesystem.syn]:

```
namespace std::filesystem{
    ...
    previous declarations
    ...

    // *30.11.15*, *suffix for* path *literals*
    inline namespace literals{
        path operator "" generic(const char* str, size_t len);
        path operator "" generic(const wchar_t* str, size_t len);
        path operator "" generic(const char16_t* str, size_t len);
        path operator "" generic(const char32_t* str, size_t len);

        path operator "" native(const char* str, size_t len);
        path operator "" native(const wchar_t* str, size_t len);
        path operator "" native(const char16_t* str, size_t len);
        path operator "" native(const char32_t* str, size_t len);

        path operator "" path(const char* str, size_t len);
        path operator "" path(const wchar_t* str, size_t len);
        path operator "" path(const char16_t* str, size_t len);
        path operator "" path(const char32_t* str, size_t len);
    }
}
```

After subclause 30.11.14 [fs.op.funcs] add a new subclause 30.11.15 [fs.filesystem.literal]:

```
path operator "" generic(const char* str, size_t len);
```

Returns: `path(str, str + len, path::generic_format)`.

```
path operator "" generic(const wchar_t* str, size_t len);
```

Returns: `path(str, str + len, path::generic_format)`.

```
path operator "" generic(const char16_t* str, size_t len);
```

Returns: `path(str, str + len, path::generic_format)`.

```
path operator "" generic(const char32_t* str, size_t len);
```

Returns: `path(str, str + len, path::generic_format)`.

```
path operator "" native(const char* str, size_t len);
```

Returns: `path(str, str + len, path::native_format)`.

```
path operator "" native(const wchar_t* str, size_t len);
```

Returns: `path(str, str + len, path::native_format)`.

```
path operator "" native(const char16_t* str, size_t len);
```

Returns: `path(str, str + len, path::native_format)`.

```
path operator "" native(const char32_t* str, size_t len);
```

Returns: `path(str, str + len, path::native_format)`.

```
path operator "" path(const char* str, size_t len);
```

Returns: `path(str, str + len)`.

```
path operator "" path(const wchar_t* str, size_t len);
```

Returns: `path(str, str + len)`.

```
path operator "" path(const char16_t* str, size_t len);
```

Returns: `path(str, str + len)`.

```
path operator "" path(const char32_t* str, size_t len);
```

Returns: `path(str, str + len)`.

Acknowledgments

Special thanks to Zhihao Yuan for his helpful review for this proposal.